



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**



Algorithm Analysis

Introduction

C

P

C

S

2

0

4

3

Algorithm Analysis - Introduction

- Efficiency
 - Max # of 1's
 - $O(n)$
 - Linear Search
 - $O(n)$
 - Binary Search
 - $O(\log_2 n)$
 - Sorted List Matching Problems
 - $O(n)$

C

P

C

S

2

0

4

4

Algorithm Analysis - **Introduction**

- We will use Order Notation to approximate two things about algorithms:
 - How much time they take
 - How much memory (space) they use
- Note:
 - It is nearly impossible to figure out the exact amount of time an algorithm will take
 - Each algorithm gets translated into smaller and smaller machine instructions
 - Each of these instructions take various amounts of time to execute on different computers

C

P

C

S

2

0

4

5

Algorithm Analysis - **Introduction**

- We want to judge algorithms **independent** of their implementation
- Specifically
 - we want to know how the **performance** of an algorithm **responds to changes** in problem size
- And, rather than figure out an algorithm's exact running time
 - We only want **an approximation**
 - **Big-O** approximation

Algorithm Analysis - Introduction

- Big – O Approximation
 - After analyzing an algorithm, we end up with
 - Polynomial equation/function

$$f(n) = 4n^2 + 3n + 10 = O(n^2)$$

- Growth of the function

n	4n ²	3n	10
1	4	3	10
10	400	30	10
100	40,000	300	10
1000	4,000,000	3,000	10
10,000	400,000,000	30,000	10
100,000	40,000,000,000	300,000	10
1,000,000	4,000,000,000,000	3,000,000	10

Algorithm Analysis - Introduction

- Big – O Approximation $f(n) = 4n^2 + 3n + 10$
 - Efficiency is approximated by **highest order term**
 - $4n^2$
 - Eliminate all **lower order** terms
 - $3n + 10$
 - Ignore all **coefficient constants** too
 - 4 (from $4n^2$)
 - Hence

$$f(n) = 4n^2 + 3n + 10 = O(n^2)$$

C

P

C

S

2

0

4

Algorithm Analysis - Introduction

- Big – O Approximation

- $f(n) = 4n^2 + 3n + 10$

- $O(n^2)$

- $f(n) = 76,756,234n^2 + 427,913n + 7$

- $O(n^2)$

- $f(n) = 74n^8 - 62n^5 - 71562n^3 + 3n^2 - 5$

- $O(n^8)$

- $f(n) = 42n^4 * (12n^6 - 73n^2 + 11)$

- $O(n^{10})$

- $f(n) = 75n * \log n - 415$

- $O(n * \log n)$

C

P

C

S

2

0

4

Big-Oh - Introduction

- $4n^2 + 3n + 10 = O(n^2)$
- We have one function:
 - $f(n) = 4n^2 + 3n + 10$
- Let us make a second function, $g(n)$
 - $g(n) = n^2$
- So now we have
 - $f(n) = O(g(n))$

C

P

C

S

2

0

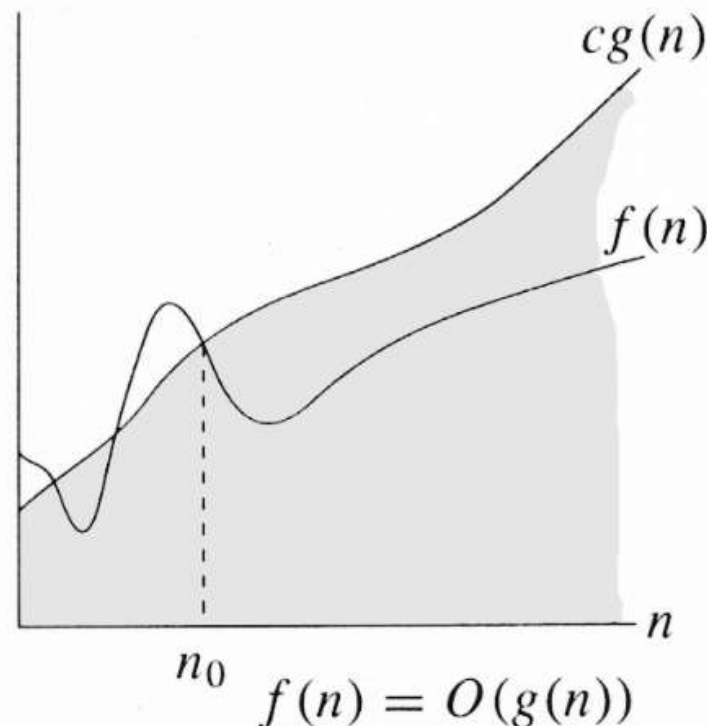
4

10

Big-Oh – Introduction (formal definition)

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$

- $c * g(n)$ is an **upper bound** on the value of $f(n)$
- Such that
 - $f(n) \leq c * g(n)$



C

P

C

S

2

0

4

Common Order – Introduction

- Complexity Classes
- The best running time
 - $O(1)$
- The Worst running time
 - $O(n!)$

Function	Name
1	Constant
$\log n$	Logarithmic
n	Linear
$n \log n$	Poly-log
n^2	Quadratic
n^3	Cubic
2^n	Exponential
$n!$	Factorial

C

P

C

S

2

0

4

Complexity Classes – $O(1)$

- It is a notation for “constant work”
- An example would be finding the smallest element in a sorted array
 - There’s nothing to search for here
 - The smallest element is always at the beginning of a sorted array
 - So this would take $O(1)$ time

C

P

C

S

2

0

4

Complexity Classes – $O(n)$

- It means that the work changes in a way that is proportional to n
- Example:
 - If the input size doubles, the running time also doubles
- It is a notation for “work grows at a linear rate”
- You usually can't really do a lot better than this for most problems we deal with

C

P

C

S

2

0

4

Complexity Classes – $O(n^k)$

- $O(n^2)$ or “Order n^2 ”: Quadratic time
 - If input size doubles, running time increases by a factor of n^2
- $O(n^3)$ or “Order n^3 ”: Cubic time
 - If input size doubles, running time increases by a factor of n^3
- $O(n^k)$: Other polynomial time
 - Should really try to avoid high order polynomial running times

C

P

C

S

2

0

4

Complexity Classes – **O(Exponential)**

- **$O(2^n)$** or “Order 2^n ”: **Exponential time**
 - more theoretical rather than practical interest because they cannot reasonably run on typical computers for even for moderate values of n .
 - Input sizes bigger than 40 or 50 become unmanageable
 - Even on faster computers
- **$O(n!)$** : even worse than exponential!
 - Input sizes bigger than 10 will take a long time

C

P

C

S

2

0

4

Complexity Classes – **$O(\text{Logarithmic})$**

- **$O(n \log n)$** :
 - Only slightly worse than $O(n)$ time
 - And $O(n \log n)$ will be much less than $O(n^2)$
 - This is the running time for the better sorting algorithms we will go over (later)
- **$O(\log n)$** or “Order $\log n$ ”: **Logarithmic time**
 - If input size doubles, running time increases ONLY by a constant amount
 - **any algorithm that halves the data** remaining to be processed on each iteration of a loop will be an **$O(\log n)$ algorithm.**

Algorithm Analysis

Some Example

C

P

C

S

2

0

4

Algorithm Analysis - Example - 01

- The number of operations executed by these loops is the sum of the individual loop operations.
- The first loop has $n/2$ operations
- The second loop has n^2 operations
- Not nested, So we simply ADD their operations
- The total number of operations would be
 - $T(n) = n/2 + n^2$

```
for (k = 1; k <= n/2; k++) {  
    sum = sum + 5;  
}  
for (j = 1; j <= n*n; j++) {  
    delta = delta + 1;  
}
```

In Big-O terms, we can express the number of operations as $O(n^2)$

C

P

C

S

2

0

4

Algorithm Analysis - Example - 02

- The outer loop runs **n times**
- for each of those n times, the inner loop also runs **n times**
- So we have **n operations per iteration** of OUTER loop
- Finally, we have the number of **operations**
 - $T(n) = n * n$

```
int func1(int n) {  
    int i, j, x = 0;  
    for (i = 1; i <= n; i++) {  
        for (j = 1; j <= n; j++) {  
            x++;  
        }  
    }  
    return x;  
}
```

In Big-O terms, we can express the number of operations as **$O(n^2)$**

C

P

C

S

2

0

4

Algorithm Analysis - **Example - 03**

- The loops are **NOT nested**
- First loop runs **n times**
- Second loop runs **n times**
- Total runtime is on the order of
 - **$T(n) = n + n = 2n$**

```
int func3(int n) {  
    int i, x = 0;  
    for (i = 1; i <= n; i++)  
        x++;  
    for (i = 1; i <= n; i++)  
        x++;  
    return x;  
}
```

In Big-O terms, we can express the number of operations as **$O(n)$**

C

P

C

S

2

0

4

Algorithm Analysis - Example - 04

- Initially n
- After first iteration
 - $n/2$
- After 2nd iteration
 - $n/4$
- After 3rd iteration
 - $n/8$
- After "k" iteration
 - $n/2^k$
- Algorithm ends when
 - $n/2^k = 1$
- So $k = \log_2 n$

```
int func4(int n) {  
    while (n > 0) {  
        printf("%d", n%2);  
        n = n/2;  
    }  
}
```

In Big-O terms, we can express the number of operations as $O(\log_2 n)$

Algorithm Analysis - Example - 05

```
int func5(int[][] array, int n) {  
    int i = 0, j = 0;  
    while (i < n) {  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
    return j;  
}
```

- Outer loop can never run more than **n times**
- Inner loop can never run more than **n times**
- The MOST number of times these two statements can run (combined) is **2n times**
- we approximate the running time as **O(n)**

Algorithm Analysis - Example - 06

```
int func6(int[][] array, int n) {  
    int i = 0, j;  
    while (i < n) {  
        j = 0;  
        while (j < n && array[i][j] == 1)  
            j++;  
        i++;  
    }  
    return j;  
}
```

- Outer loop will run **n times**
- Each outer iteration, Inner loop will run **n times**
- The MOST number of times these two statements can run **n*n times**
- we approximate the running time as **$O(n^2)$**

Algorithm Analysis - Example - 07

```
int func7(int A[], int sizeA, int B[], int sizeB) {  
    int i, j;  
    for (i = 0; i < sizeA; i++)  
        for (j = 0; j < sizeB; j++)  
            if (A[i] == B[j])  
                return 1;  
    return 0;  
}
```

- the runtime here is **NOT in terms of n**
- The outer loop runs **sizeA times**
- For each iteration of outer loop, the inner loop runs **sizeB times**
- So this algorithm runs **sizeA*sizeB** times
- We approximate the running time as **O(sizeA*sizeB)**

C

P

C

S

2

0

4

Algorithm Analysis - Example - 08

```
int func8(int A[], int sizeA, int B[], int sizeB) {  
    int i, j;  
    for (i = 0; i < sizeA; i++) {  
        if (binSearch(B, sizeB, A[i]))  
            return 1;  
    }  
    return 0;  
}
```

- the runtime here is **NOT in terms of n**
- The outer loop runs **sizeA times**
- For each iteration of outer loop, the inner loop runs $\lg(\text{sizeB})$ times
- So this algorithm runs **sizeA * $\lg(\text{sizeB})$** times
- We approximate the running time as **$O(\text{sizeA} * \lg(\text{sizeB}))$**

Algorithm Analysis

Practical Problems

C

P

C

S

2

0

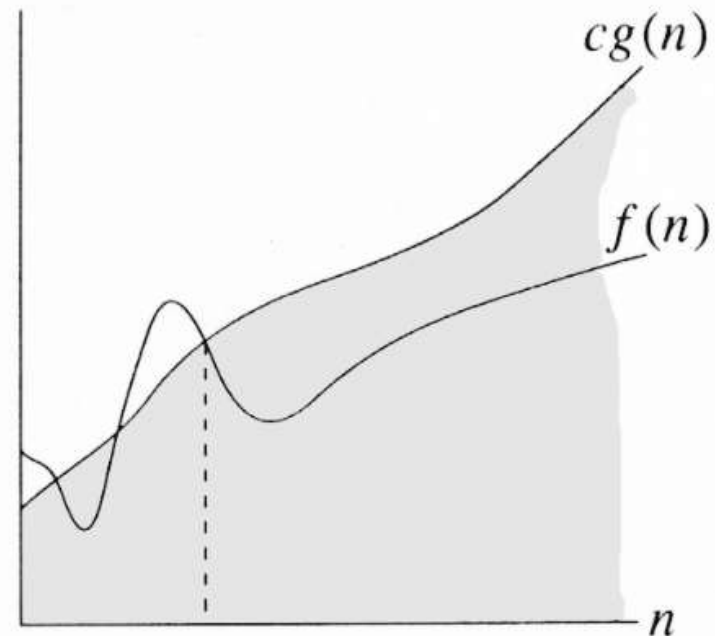
4

27

Big-Oh – Introduction (formal definition)

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$

- $c * g(n)$ is an **upper bound** on the value of $f(n)$
- Such that
 - $f(n) \leq c * g(n)$



$$f(n) = O(g(n))$$

C

P

C

S

2

0

4

Practical Problems – Example 01

- An algorithm **A** runs in **$O(n)$** time
- For input size of **10**, the algorithm takes **2** milliseconds
- How much time it will take for input size of **500**?
- $T(n) = O(n)$
 - $T(n) = c * n$
- $n = 10, T(10) = 2\text{ms}$
 - $T(10) = c * 10$
 - $2 = c * 10$
 - $c = 1/5$
- $c = 1/5, n = 500, T(500) = ?$
 - $T(500) = (1/5) * 500$
 - $T(500) = 100 \text{ ms}$

C

P

C

S

2

0

4

Practical Problems – **Example 02**

- An algorithm **A** runs in **$O(n^2)$** time
 - $T(n) = O(n^2)$
 - $T(n) = c * n^2$
- For input size of **4**, the algorithm takes **10** milliseconds
 - $n = 4, T(4) = 10\text{ms}$
 - $T(4) = c * 4^2$
 - $10 = c * 16$
 - $c = 10/16$
- How much time it will take for input size of **16**?
 - $c = 10/16, n = 16, T(16) = ?$
 - $T(16) = (10/16) * 16^2$
 - $T(16) = 160\text{ ms}$

C

P

C

S

2

0

4

Practical Problems – Example 03

- An algorithm **A** runs in **$O(\log_2 n)$** time
 - For input size of **16**, the algorithm takes **28** milliseconds
 - How much time it will take for input size of **64**?
- $T(n) = O(\log_2 n)$
 - $T(n) = c * \log_2 n$
 - $n = 16, T(16) = 28\text{ms}$
 - $T(16) = c * \log_2 16$
 - $28 = c * 4$
 - $c = 7$
 - $c = 7, n = 64, T(64) = ?$
 - $T(64) = 7 * \log_2 64$
 - $T(64) = 7 * 6$
 - $T(64) = 42 \text{ ms}$

C

P

C

S

2

0

4

Practical Problems – Example 04

- An algorithm **D** runs in **$O(\log_2 n)$** time
 - For input size of **8**, the algorithm takes **2** milliseconds
 - How much time it will take for input size of **64**?
- $T(n) = O(\log_2 n)$
 - $T(n) = c * \log_2 n$
 - $n = 8, T(8) = 2\text{ms}$
 - $T(8) = c * \log_2 8$
 - $2 = c * 3$
 - $c = 2/3$
 - $c = 2/3, n = 64, T(64) = ?$
 - $T(64) = 2/3 * \log_2 64$
 - $T(64) = 2/3 * 6$
 - $T(64) = 4\text{ ms}$

C

P

C

S

2

0

4

Practical Problems – Example 05

- An algorithm **A** runs in **$O(n^2)$** time
- For input size of **10**, the algorithm takes **30** milliseconds
- What will be the problem size if it takes **120**ms to execute?
- $T(n) = O(n^2)$
 - $T(n) = c * n^2$
- $n = 10, T(10) = 30\text{ms}$
 - $T(10) = c * 10^2$
 - $30 = c * 100$
 - $c = 30/100 = 3/10$
- $c = 3/10, n = ?, T(n)=120$
- $120 = (3/10)*n^2$
- $n^2 = 1200/3 = 400$
- $n = 20$



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). (Quran 3 : 173)